Data Structure and Summarisation

Lecture 2: Relational Theory and Descriptive Statistics



GOLDEN GATE UNIVERSITY

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Lesson 2 – Relational Theory; Descriptive Statistics

Outcomes and agenda

1

Define
relation, tuple, attribute, schema, instance and keys

2

Apply
select, project, union and join to small tables

3

Compute
mean, median, variance, standard deviation, quartiles

4

Interpret
histograms, boxplots and outliers

KEY IDEA	**Part A: relational theory. Part B: descriptive statistics.**

Why the relational model

Codd's idea	1970

- Store data as tables: simple and uniform
- Describe data by its meaning, not by file layout
- Use set theory and logic to define queries
- Separate the logical view from physical storage

What it gives us	Today

- Most databases in industry use it (SKS Part One)
- Precise query languages: algebra and SQL
- Keys and constraints that protect integrity
- A base for optimisation: equivalent expressions

KEY IDEA **A relation is a table, and a query is an operation that takes tables and returns a table.**

Structure: relation, tuple, attribute, domain

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000

instructor: a relation with 4 attributes and 5 tuples

- Relation: a table (a set of tuples)
- Tuple: one row, one real-world fact
- Attribute: a named column (id, name, ...)
- Domain: the allowed values of an attribute; values are atomic
- NULL: a special value meaning unknown or absent

KEY IDEA **Order of rows does not matter: a relation is a set (SKS 2.1).**

Schema vs instance

Schema	The design

- Names the relation and its attributes with domains
- instructor(id, name, dept_name, salary)
- Changes rarely, by deliberate design
- Like the type of a variable

Instance	A snapshot

- The rows stored at one moment
- Changes with every insert, update and delete
- Must always satisfy the schema and its constraints
- Like the value of a variable

KEY IDEA **Design the schema carefully; the instance changes every day.**

Keys: identifying tuples

Term	Meaning	Example in instructor
Superkey	Any set of attributes that identifies a tuple uniquely	{id}, {id, name}
Candidate key	A superkey with no unnecessary attribute (minimal)	{id}
Primary key	The candidate key chosen by the designer	id
Foreign key	Attributes that refer to the primary key of another relation	dept_name refers to department

KEY IDEA	**A key is a rule about the data, enforced by the DBMS, not a convention.**

Foreign keys and referential integrity

instructor (references department)

name	dept_name	salary
Srinivasan	Comp. Sci.	65000
Wu	Finance	90000
Einstein	Physics	95000
Gold	Physics	87000



FK refers to PK

department (primary key: dept_name)

dept_name	building	budget
Comp. Sci.	Taylor	100000
Finance	Painter	120000
Physics	Watson	70000

- Every dept_name in instructor must exist in department (referential integrity)
- Inserting an instructor in a department that does not exist is rejected
- Deleting a department that still has instructors is rejected, or can cascade

KEY IDEA	**Foreign keys connect relations; referential integrity keeps the connections valid.**

Select (σ): choose rows

σ dept_name = 'Physics' (instructor)

instructor

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000



σ select

Result: rows that satisfy the predicate

id	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

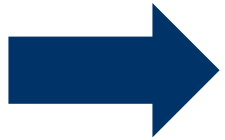
KEY IDEA **Select keeps all attributes and filters tuples. SQL: WHERE.**

Project (π): choose columns

π name, salary (instructor)

instructor

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
22222	Einstein	Physics	95000
33456	Wu	Physics	90000
45565	Wu	Comp. Sci.	75000



π project

Result: only the chosen attributes

name	salary
Srinivasan	65000
Wu	90000
Einstein	95000
Wu	75000

KEY IDEA **Project keeps some attributes and removes duplicate rows. SQL: SELECT DISTINCT.**

Set operations and renaming

Operation	Symbol	Meaning	Requirement
Union	R ∪ S	Tuples in R or S or both	Same attributes and domains
Intersection	R ∩ S	Tuples in both R and S	Same attributes and domains
Difference	R − S	Tuples in R but not in S	Same attributes and domains
Rename	ρ x (R)	Gives a new name to a relation or attributes	None

KEY IDEA **Relations are sets, so the usual set operations apply when the schemas are compatible.**

Join (⋈): combine relations

π name, dept_name, building (instructor ⋈ department)

instructor

name	dept_name	salary
Srinivasan	Comp. Sci.	65000
Wu	Finance	90000
Einstein	Physics	95000
Gold	Physics	87000

department

dept_name	building	budget
Comp. Sci.	Taylor	100000
Finance	Painter	120000
Physics	Watson	70000



⋈ join

Result: tuples matched on dept_name

name	dept_name	building
Srinivasan	Comp. Sci.	Taylor
Wu	Finance	Painter
Einstein	Physics	Watson
Gold	Physics	Watson

KEY IDEA — **Join pairs tuples that agree on shared attributes. SQL: JOIN ... ON or NATURAL JOIN.**

Composing operations

RELATIONAL ALGEBRA AND SQL

```
Algebra:
π name, salary (
  σ dept_name = 'Physics' and salary > 90000
    (instructor))

SQL:
SELECT name, salary FROM instructor
WHERE dept_name = 'Physics'
  AND salary > 90000;
-- Result: (Einstein, 95000)
```

Why composition matters

- Every operator takes relations and returns a relation
- So operators nest into expression trees
- The optimiser rewrites trees into cheaper equivalents
- SQL is a practical form of the algebra

KEY IDEA **Closure under operations is what makes optimisation possible.**

Checkpoint

- In instructor(id, name, dept_name, salary), name two superkeys and one candidate key.
- Write the algebra for: names of instructors in Finance.
- Why does inserting an instructor in department "Law" fail?
- How many tuples does π dept_name (instructor) return for our 5-row sample?

KEY IDEA	**Answers: {id} and {id,name}; π name (σ dept_name='Finance' (instructor)); referential integrity; 3.**

Part B: populations, samples, data types

Population and sample

Navidi 1.1

- Population: the whole collection of interest
- Sample: the part we actually observe
- Simple random sample: every member has an equal chance
- Descriptive statistics summarise the sample

Types of data

What we compute

- Numerical: salary, score, delay in ms
- Categorical: department, status, protocol
- Different types need different summaries
- Tables in a DBMS hold both types

KEY IDEA

Statistics begins by asking: what is the population, and how was the sample drawn?

Centre: mean, median, mode

52	55	58	60	61	240

Mean	Median	Mode
$\bar{x} = \Sigma x_i / n$ **= 526 / 6 ≈ 87.7** Uses every value; pulled by the outlier	**Middle of the sorted data** **= (58 + 60) / 2 = 59** Resistant to outliers	**Most frequent value** **None: all values differ** Useful for categorical data

KEY IDEA **Mean and median answer "what is typical?" in different ways; the outlier separates them.**

Outliers and robustness



- **Mean with 240: 87.7**
- Mean without 240: 57.2
- **Median with 240: 59**
- Median without 240: 58
- The mean moved by 30; the median moved by 1
- Always check outliers: error, or real extreme?

KEY IDEA **A statistic that barely moves when one value changes is called resistant (robust).**

Spread: variance and standard deviation

x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$
2	-3	9
4	-1	1
4	-1	1
4	-1	1
5	0	0
5	0	0
7	2	4
9	4	16
$\Sigma = 40$	$\Sigma = 0$	$\Sigma = 32$

- **Mean: $\bar{x} = 40 / 8 = 5$**
- Sample variance: $s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1) = 32 / 7 \approx 4.57$
- Sample standard deviation: $s = \sqrt{4.57} \approx 2.14$
- Dividing by n gives 4 and 2: the population formulas
- Range = max − min = 9 − 2 = 7
- Standard deviation has the same units as the data

KEY IDEA **Spread tells us how far values typically lie from the centre.**

Quartiles, IQR and the outlier rule

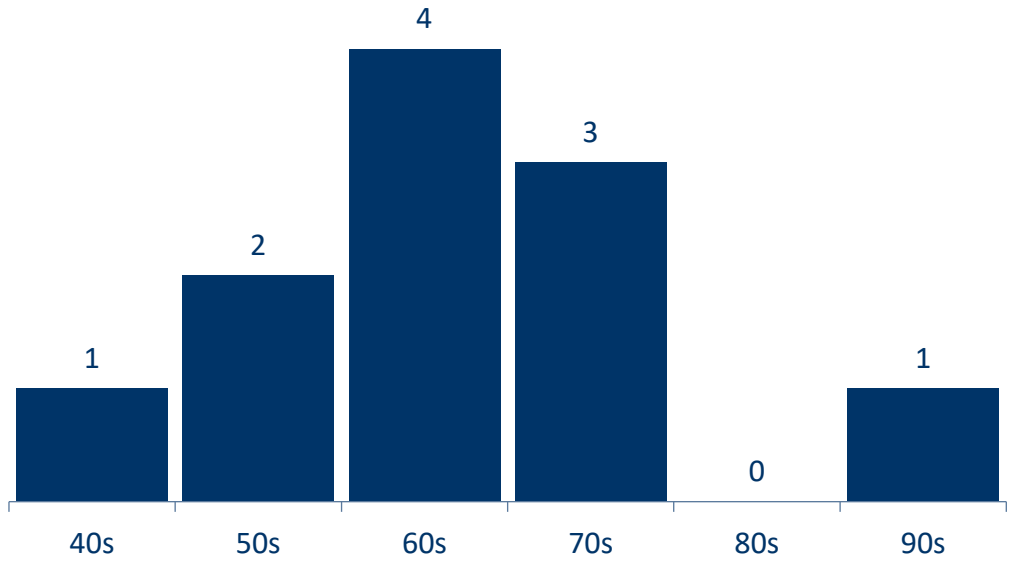
Exam scores, sorted (n = 11): 45 52 58 60 61 63 65 70 72 78 95

45	58	63	72	95
Min	Q1	Median	Q3	Max

- IQR = Q3 − Q1 = 72 − 58 = 14 (spread of the middle half)
- Fences: Q1 − 1.5 × IQR = 37 and Q3 + 1.5 × IQR = 93
- 95 lies above 93, so it is flagged as an outlier
- Software using interpolation reports Q1 = 59, Q3 = 71 (IQR 12, fences 41 and 89): same verdict

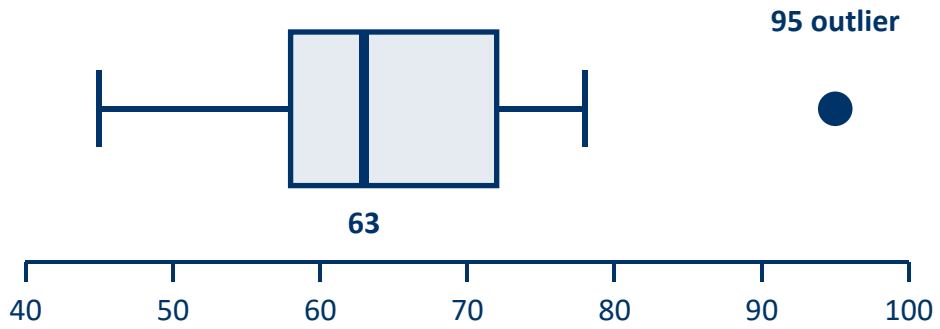
KEY IDEA **Quartile methods differ slightly between textbooks and software; state which one you use.**

Graphical summaries: histogram and boxplot



Histogram: counts of the 11 scores per 10-point bin

Boxplot of the same scores



- Histogram shows the shape: mostly 60s and 70s
- Boxplot shows centre, spread and outliers

KEY IDEA **Always plot the data: numbers can hide shape and outliers.**

From the DBMS: SQL aggregates

SQL (PostgreSQL)

```
SELECT COUNT(*)            AS n,
       AVG(salary)           AS mean,
       STDDEV_SAMP(salary)  AS sd,
       MIN(salary), MAX(salary),
       percentile_cont(0.5) WITHIN GROUP
         (ORDER BY salary) AS median
FROM instructor;
```

Why compute in the DBMS?

- Statistics can be computed inside the DBMS
- Only the summary crosses the network
- STDDEV_POP uses n; STDDEV_SAMP uses n − 1
- percentile_cont interpolates (PostgreSQL)

KEY IDEA **Compute close to the data: summarise in the DBMS, then plot in Python or R.**

Summary and exit question

WRAP-UP

Exit question

- A salary column has a few very high executive salaries. Which centre measure do you report, and why?
- Structure: relations, keys and algebra. Summary: centre, spread, quartiles, plots.
- Lab next: set up a DBMS, connect over the network, then compute and compare statistics in SQL, Python and R.

KEY IDEA **Structure the data first, then summarise it; both steps depend on the DBMS.**